SSR VỚI DATABASE VÀ ORM

MSc. Ngô Ngọc Đăng Khoa

SSR VỚI DATABASE VÀ ORM

NỘI DUNG BUỔI HỌC

01 Câu hỏi và mental model

02 Supabase, ORM và truyền dữ liệu giữa các trang

03 Ghi dữ liệu và xử lý lỗi

04 Bài tập CRUD món ăn

05 Nghiệm thu và tổng kết

CÂU HỎI

Server tổ chức các trang và xử lý **CRUD** thế nào để UI luôn phản ánh đúng dữ liệu trong database?

MỤC TIÊU

Nối ứng dụng với database Supabase

01

So sánh SQL thô, query builder, ORM và truy vấn qua model

02

Chọn `params`, `query` hay `form` để truyền dữ liệu giữa các trang

03

Dùng `POST`, `PATCH`, `DELETE` để ghi dữ liệu rồi redirect, không dùng `GET`

04

THUẬT NGỮ CỐT LÕI

THUẬT NGỮ	NGHĨA DÙNG TRONG BLOCK
ORM	Thư viện cho phép truy vấn database bằng model và hàm thay cho chuỗi SQL
Model	Khai báo một bảng: tên trường, kiểu dữ liệu, giá trị mặc định
Path param, query param	Id của bản ghi nằm trong đường dẫn; điều kiện hiển thị nằm sau dấu `?`
Post/Redirect/Get	Ghi bằng POST, rồi server trả redirect để browser tải trang bằng GET

MENTAL MODEL

NỐI ỨNG DỤNG VỚI DB (SUPABASE)

01

TẠO PROJECT: Đặt mật khẩu database khi tạo project Supabase

02

LẤY CHUỖI KẾT NỐI: Nút Connect, chọn Session pooler, cổng 5432

03

LƯU CHUỖI KẾT NỐI: Gán cho `DATABASE_URL` trong `.env`, thêm `.env` vào `.gitignore`

04

KIỂM TRA KẾT NỐI: Chạy một truy vấn đọc đơn giản từ ứng dụng

CHUỖI KẾT NỐI KHÔNG RỜI KHỎI SERVER

`DATABASE_URL` chỉ nằm trong `.env` trên server

01

Không commit `.env`, không dán vào prompt, chat hay ảnh chụp

02

Không truyền `process.env` hoặc client ORM vào `res.render`

03

CÁC GIẢI PHÁP TRUY CẬP DB TỪ ỨNG DỤNG

```
// 1. SQL thô với pg
const { rows } = await pool.query("SELECT * FROM dishes WHERE id = $1", [id]);

// 2. Query builder (knex)
const dish = await knex("dishes").where({ id }).first();

// 3. ORM (Prisma)
const dish = await prisma.dish.findUnique({ where: { id } });
```

- Cả ba lấy một dòng của bảng `dishes`; với `pg`, dòng đó là `rows[0]`
- Giá trị `id` truyền dạng tham số, không nối chuỗi SQL
- Cả ba chạy trên server và dùng chuỗi kết nối ở server

TRADE-OFF

GIẢI PHÁP	ĐƯỢC	MẤT
SQL thô (`pg`)	Thấy chính xác SQL nào chạy; không thêm thư viện	Lặp lại nhiều; sai tên cột chỉ lộ khi chạy
Query builder (`knex`)	Ghép điều kiện động bằng hàm, không nối chuỗi SQL	Vẫn phải hiểu SQL
ORM (Prisma)	Model và object thay cho bảng và dòng; có quan hệ, migration	Thêm khái niệm; không thấy được SQL sinh ra ngay

CODE-FIRST VÀ DB-FIRST

Code-first

- Khai báo model trong **mã nguồn**; công cụ tạo hoặc cập nhật CSDL theo model
- Model trong repository là nguồn của schema
- Thường dùng khi bắt đầu dự án từ đầu

DB-first

- Bảng đã có trong database; công cụ đọc schema rồi sinh model
- Database là nguồn của schema
- Thường dùng khi database đã tồn tại hoặc do nhóm khác quản lý

ORM THEO NGÔN NGỮ VÀ FRAMEWORK

STACK	ORM THƯỜNG DÙNG	VỚI FRAMEWORK
Node.js, Express	Prisma, Sequelize, TypeORM, Drizzle	Cài thêm
Python, Django	Django ORM	Đi kèm
Python, Flask hoặc FastAPI	SQLAlchemy	Cài thêm
PHP, Laravel	Eloquent	Đi kèm
Java, Spring	Hibernate qua Spring Data JPA	Cài thêm
.NET, ASP.NET Core	Entity Framework Core	Cài thêm
Ruby, Rails	Active Record	Đi kèm

LỆNH CỦA TỪNG ORM

ORM	CODE-FIRST	DB-FIRST
Prisma	`prisma migrate dev`, `prisma db push`	`prisma db pull`
Django ORM	`makemigrations`, `migrate`	`inspectdb`
Entity Framework	`dotnet ef migrations add`	`dotnet ef dbcontext scaffold`

VÍ DỤ: MODEL DISH

```
model Dish {
  id          Int      @id @default(autoincrement())
  name        String
  description String?
  price       Int
  category    String?
  available   Boolean  @default(true)
  createdAt   DateTime @default(now())

  @@map("dishes")
}
```

- Mỗi model là một bảng, mỗi trường là một cột
- `id` do database sinh; `String?` cho phép null
- Code-first: `npx prisma db push` tạo bảng trên Supabase từ model

ĐỌC DỮ LIỆU QUA MODEL

```
// danh sách, có tìm theo tên
const dishes = await prisma.dish.findMany({
  where: { name: { contains: q, mode: "insensitive" } },
  orderBy: { id: "asc" },
});

// một món theo id, null nếu không có
const dish = await prisma.dish.findUnique({ where: { id } });
```

- `findUnique` trả `null`: handler phải trả 404, không render tiếp
- `id` đến từ URL là chuỗi: chuyển thành số và kiểm tra trước khi truy vấn

GHI DỮ LIỆU QUA MODEL

```
const created = await prisma.dish.create({ data });
await prisma.dish.update({ where: { id }, data });
await prisma.dish.delete({ where: { id } });
```

- `data` là object đã qua validate, không phải `req.body` nguyên bản
- Thiếu `await`: handler redirect khi ghi chưa xong, hoặc báo thành công khi ghi thất bại
- `created.id` là id database vừa sinh, dùng cho redirect

XÉT CHỨC NĂNG: XEM DANH SÁCH VÀ CHI TIẾT MÓN ĂN

01

TRANG DANH SÁCH: `GET /dishes` đọc các món từ database rồi hiển thị

02

LINK: Mỗi món có link dạng `/dishes/2`, id của món nằm trong link

03

TRANG CHI TIẾT: `GET /dishes/2` cần biết món nào, nên đọc id từ URL

04

DATABASE: Dùng id để lấy đúng một món rồi hiển thị

TRANG DANH SÁCH: LINK MANG ID

```
router.get("/dishes", async (req, res) ⇒ {
  const dishes = await prisma.dish.findMany({ orderBy: { id: "asc" } });
  res.render("dishes/index", { dishes });
});
```

```
<% dishes.forEach(d ⇒ { %>
  <li><a href="/dishes/<%= d.id %>"><%= d.name %></a></li>
<% }) %>
```

- Template tạo mỗi link từ `d.id`: món có id 2 cho link `/dishes/2`
- Phần `2` nằm ngay trong đường dẫn, gọi là path param

TRANG CHI TIẾT: LẤY ID TỪ URL

```
router.get("/dishes/:id", async (req, res) ⇒ {
  const id = Number(req.params.id);   // "2" trong URL thành số 2
  const dish = await prisma.dish.findUnique({ where: { id } });
  res.render("dishes/show", { dish });
});
```

- `:id` trong route khớp phần cuối đường dẫn; giá trị nằm trong `req.params.id`
- Handler dùng id để đọc đúng một món từ database, rồi truyền `{ dish }` cho template

TÌM MÓN: ĐIỀU KIỆN ĐI QUA QUERY

```
<form method="GET" action="/dishes">
  <input name="q"> <button>Tìm</button>
</form>
```

```
const q = req.query.q || "";   // "phở" từ /dishes?q=phở
const dishes = await prisma.dish.findMany({
  where: { name: { contains: q, mode: "insensitive" } },
});
```

- Form dùng GET nên từ khóa nằm sau dấu `?` trong URL: `/dishes?q=phở`
- Path param cho biết bản ghi nào, query param cho biết hiển thị thế nào

THÊM MÓN: NỘI DUNG ĐI QUA BODY

```
<form method="POST" action="/admin/dishes">
  <input name="name"> <input name="price"> <button>Thêm</button>
</form>
```

```
router.post("/admin/dishes", async (req, res) ⇒ {
  const data = { name: req.body.name, price: Number(req.body.price) };
  await prisma.dish.create({ data });
});
```

- Dữ liệu form nằm trong body, không nằm trong URL; `name` của input là khóa
- `express.urlencoded` đọc body vào `req.body`

TÓM TẮT

NƠI	DÙNG CHO	VÍ DỤ
`req.params`	Cho biết bản ghi nào	`/dishes/2` cho `{ id: "2" }`
`req.query`	Điều kiện hiển thị, tìm kiếm	`/dishes?q=phở`
`req.body`	Nội dung cần ghi	Form POST: `name=...&price=...`

TRUYỀN ID, ĐỌC LẠI TỪ DATABASE

Không nên

- Gửi cả object món ăn từ trang này sang trang khác
- Trang đích dùng dữ liệu như lúc trang nguồn render

Nên

- Chỉ gửi `id` trong URL
- Trang đích đọc lại từ database ở mỗi request

LỖI THƯỜNG GẶP KHI ĐỌC DỮ LIỆU TỪ REQUEST

`req.params.id` là chuỗi `"2"`, không phải số

01

`req.query.q` thành mảng khi URL là `?q=a&q=b`

02

Giá trị form là chuỗi, kể cả `type="number"`

03

Input thiếu `name` không xuất hiện trong request

04

CRUD VỚI GET, POST, PATCH, DELETE

THAO TÁC	METHOD	URL	KẾT QUẢ
Danh sách	`GET`	`/admin/dishes`	200
Mở form thêm	`GET`	`/admin/dishes/new`	200
Thêm	`POST`	`/admin/dishes`	303 sang chi tiết, hoặc 400
Mở form sửa	`GET`	`/admin/dishes/:id/edit`	200 hoặc 404
Sửa	`PATCH`	`/admin/dishes/:id`	303, 400 hoặc 404
Xác nhận xóa	`GET`	`/admin/dishes/:id/delete`	200 hoặc 404
Xóa	`DELETE`	`/admin/dishes/:id`	303 sang danh sách, hoặc 404

HTTP METHOD

GET: hiển thị

- Browser, công cụ tìm kiếm và tiện ích tải trước có thể tự gửi GET
- Link `/admin/dishes/2/xoa-luon` có thể xóa dữ liệu khi không ai bấm

POST, PATCH, DELETE: ghi dữ liệu

- POST thêm bản ghi, PATCH sửa, DELETE xóa
- Trang xác nhận xóa là GET; thao tác xóa là DELETE

PATCH VÀ DELETE TỪ FORM HTML

```
<form method="POST">
  <button formaction="/admin/dishes/2?_method=PATCH">Lưu</button>
  <button formaction="/admin/dishes/2?_method=DELETE">Xóa</button>
</form>
```

```
app.use(methodOverride("_method")); // POST kèm ?_method=DELETE được xử lý như DELETE
app.patch("/admin/dishes/:id", updateDish);
app.delete("/admin/dishes/:id", deleteDish);
```

- `formaction` đặt URL riêng cho từng button, kèm `?_method=PATCH` hoặc `?_method=DELETE`
- Browser vẫn gửi POST; `method-override` đổi thành PATCH hoặc DELETE

POST/REDIRECT/GET

POST/REDIRECT/GET GIẢI QUYẾT GÌ

Giải quyết

- Reload sau khi ghi chỉ lặp lại GET
- URL trên thanh địa chỉ là trang kết quả, gửi được cho người khác
- Back không quay về một POST đã xử lý

Không giải quyết

- Bấm gửi hai lần trước khi response đầu về vẫn có thể tạo hai bản ghi
- Hai tab hoặc hai người gửi cùng nội dung vẫn tạo hai bản ghi
- Chặn trùng thật sự cần ràng buộc ở database

MÃ LỖI

LOẠI	VÍ DỤ	STATUS	SERVER LÀM
Input sai	`name` trống, `price` âm	`400`	Không ghi database; render lại form kèm lỗi và giá trị đã nhập
Không tìm thấy	`/dishes/999`, `/dishes/abc`	`404`	Trang 404 có link quay về
Lỗi hệ thống	Mất kết nối database	`500`	Ghi chi tiết vào log, trả trang lỗi chung

LỖI THƯỜNG GẶP KHI GHI DỮ LIỆU

Dùng link GET để xóa hoặc sửa dữ liệu 01

Render HTML ngay trong response của POST, Reload tạo bản ghi trùng 02

Hiển thị `err.message` của database cho người dùng 03

Redirect khi chưa `await` xong, hoặc báo thành công khi ghi thất bại 04

BÀI TẬP

XÂY ỨNG DỤNG NHÀ HÀNG

a) Trang người dùng xem món ăn,

b) Trang quản trị CRUD món ăn, đầu bếp và nguyên liệu.

MÔ TẢ

Project Express và EJS khởi đầu, có `.env.example` và `.gitignore` chứa `.env`

01

Mỗi nhóm một project Supabase, `DATABASE_URL` chỉ nằm trong `.env`

02

Ba model `Dish`, `Chef`, `Ingredient`, trường gợi ý ở slide kế tiếp

03

Script seed có dữ liệu cho cả ba bảng

04

DỮ LIỆU

MODEL	TRƯỜNG GỢI Ý
`Dish` (món ăn)	`name`, `description`, `price`, `category`, `available`
`Chef` (đầu bếp)	`name`, `specialty`
`Ingredient` (nguyên liệu)	`name`, `unit`

TRANG NGƯỜI DÙNG

TRANG	URL	YÊU CẦU
Danh sách món ăn	`GET /dishes`	Tìm theo tên bằng query `q`; mỗi món có link sang trang chi tiết
Chi tiết món ăn	`GET /dishes/:id`	Hiện đủ các trường; 404 khi id sai hoặc không có

TRANG QUẢN TRỊ

01 `/admin/dishes`, `/admin/chefs`, `/admin/ingredients` đều có đủ danh sách, thêm, sửa, xóa theo bảng CRUD

02 Thêm dùng POST, sửa dùng PATCH, xóa dùng DELETE; không dùng `fetch`, AJAX hay JavaScript phía client để gửi dữ liệu

03 Input sai trả 400, giữ giá trị đã nhập; id sai hoặc không có trả 404

04 Thao tác thành công redirect 303; xóa có trang xác nhận

QUY TRÌNH LÀM BÀI

01

NỐI DATABASE: Đặt `DATABASE_URL`, khai báo ba model, chạy `db push`, seed

02

TRANG NGƯỜI DÙNG: Danh sách và chi tiết món ăn, 404

03

CRUD MÓN ĂN: Form, validate, method override, redirect sau khi ghi

04

ĐẦU BẾP VÀ NGUYÊN LIỆU: Lặp lại mẫu của món ăn, rồi đối chiếu tiêu chí nghiệm thu

KIỂM TRA MÃ NGUỒN: METHOD VÀ REQUEST

Có link hoặc route GET nào xóa hay sửa dữ liệu không 01

Sửa và xóa có route PATCH và DELETE qua method override không 02

Thao tác ghi thành công có redirect 303 không 03

Template có `fetch` hoặc AJAX không 04

KIỂM TRA MÃ NGUỒN: DỮ LIỆU VÀ BẢO MẬT

Giá trị người dùng nhập có bị nối vào raw query không

01

`req.body` có bị đưa thẳng vào `create` hoặc `update` không

02

Chuỗi kết nối có xuất hiện trong mã, log hoặc template không

03

TIÊU CHÍ NGHIỆM THU

Trang người dùng có danh sách và chi tiết món ăn; id sai hoặc không có trả 404

01

Món ăn, đầu bếp, nguyên liệu đều có danh sách, thêm, sửa, xóa ở `/admin`

02

Thêm dùng POST, sửa dùng PATCH, xóa dùng DELETE; không có `fetch` hay AJAX

03

Input sai trả 400 và giữ giá trị đã nhập; thao tác thành công trả 303

04

Xóa chỉ xảy ra sau trang xác nhận; Reload không lặp lại thao tác

05

TỔNG KẾT

Dữ liệu nằm trong database; mỗi request đọc lại nên HTML khớp với dữ liệu

01

ORM thay SQL thô bằng model, vẫn chạy trên server cùng chuỗi kết nối

02

Param cung cấp thông tin bản ghi, query cung cấp điều kiện hiển thị, body cung cấp nội dung ghi

03

GET để đọc; POST, PATCH, DELETE để ghi; redirect sau khi ghi

04

Q & A